

Lesson 16: Recursion

Recursion is defined as a function calling itself. It is in some ways similar to a loop because it repeats the same code, but it requires passing in the looping variable and being more careful. Many programming languages allow it because it can simplify some tasks, and it is often more elegant than a loop.

A simple example of recursion would be:

```
void recurse()  
{  
    recurse(); //Function calls itself  
}  
  
int main()  
{  
    recurse(); //Sets off the recursion  
}
```

This program will not continue forever, however. The computer keeps function calls on a stack and once too many are called without ending, the program will crash. Why not write a program to see how many times the function is called before the program terminates?

```
#include <iostream>  
  
using namespace std;  
  
void recurse ( int count ) // Each call gets its own count  
{  
    cout<< count <<"\n";  
    // It is not necessary to increment count since each function's  
    // variables are separate (so each count will be initialized one greater)  
    recurse ( count + 1 );  
}  
  
int main()  
{  
    recurse ( 1 ); //First function call, so it starts at one  
}
```

This simple program will show the number of times the recurse function has been called by initializing each individual function call's count variable one greater than it was previous by passing in count + 1. Keep in mind, it is not a function restarting itself, it is hundreds of functions that are each unfinished with the last one calling a new recurse function.

It can be thought of like the Russian dolls that always have a smaller doll inside. Each doll calls another doll, and you can think of the size being a counter variable that is being decremented by one.

Think of a really tiny doll, the size of a few atoms. You can't get any smaller than that, so there are no more dolls. Normally, a recursive function will have a variable that performs a similar action; one that controls when the function will finally exit. The condition where the function will not call itself is termed the base case of the function. Basically, it is an if-statement that checks some variable for a condition

(such as a number being less than zero, or greater than some other number) and if that condition is true, it will not allow the function to call itself again. (Or, it could check if a certain condition is true and only then allow the function to call itself).

A quick example:

```
void doll ( int size )
{
    if ( size == 0 )    // No doll can be smaller than 1 atom (10^0==1) so doesn't call
        return;        // Return does not have to return something, it can be used
                        // to exit a function
    doll ( size - 1 ); // Decrements the size variable so the next doll will be smaller
}
int main()
{
    doll ( 10 ); //Starts off with a large doll (its a logarithmic scale)
}
```

This program ends when size equals one. This is a good base case, but if it is not properly set up, it is possible to have an base case that is always true (or always false).

Once a function has called itself, it will be ready to go to the next line after the call. It can still perform operations. One function you could write could print out the numbers 123456789987654321. How can you use recursion to write a function to do this? Simply have it keep incrementing a variable passed in, and then output the variable...twice, once before the function recurses, and once after...

```
void printnum ( int begin )
{
    cout<< begin;
    if ( begin > 9 )        // The base case is when begin is greater than 9
        printnum ( begin + 1 ); // for it will not recurse after the if-statement
    cout<< begin;           // Outputs the second begin, after the program has
                        // gone through and output
}
```

This function works because it will go through and print the numbers begin to 9, and then as each printnum function terminates it will continue printing the value of begin in each function from 9 to begin.

This is just the beginning of the usefulness of recursion. Heres a little challenge, use recursion to write a program that returns the factorial of any number greater than 0. (Factorial is number*number-1*number-2...*1).

Hint: Recursively find the factorial of the smaller numbers first, ie, it takes a number, finds the factorial of the previous number, and multiplies the number times that factorial...have fun. :-)